Лабораторная работа № 4

Эмуляция и измерение задержек в глобальных сетях

Авдадаев Джамал Геланиевич

2026-01-01

Содержание

Цель работы 1
Задание 1
Теоретическое введение 2
Выполнение лабораторной работы 2
1. Создание и проверка топологии 2
2. Интерактивные эксперименты NETEM 4
3. Воспроизводимый эксперимент `simple-delay` 7
4. Самостоятельный эксперимент `change-delay` 13
5. Самостоятельный эксперимент `jitter-delay` 17
6. Самостоятельные эксперименты `correlation-delay` и `distribution-delay` 21
7. Повторный запуск и итоговая статистика 24
Выводы 26
Список литературы 26

Дисциплина: Моделирование сетей передачи данных
Среда: Mininet VM (Ubuntu 20.04), Python, Linux NETEM (`tc`), `ping`, Gnuplot, GNU Make
Исходный проект: network-modeling / labs/lab04/project

Цель работы

Познакомиться с инструментом NETEM и получить навыки интерактивного и воспроизводимого измерения задержки передачи пакетов и её дрожания (jitter) в виртуальной сети Mininet, имитирующей свойства глобальной сети.

Задание

1. Создать топологию из двух хостов и коммутатора в сети `10.0.0.0/8`.
2. Выполнить интерактивные опыты с постоянной задержкой, её изменением, джиттером, корреляцией и нормальным распределением задержки.
3. Реализовать воспроизводимый эксперимент `simple-delay` с автоматической записью RTT, построением PNG-графика и расчётом статистики.
4. Самостоятельно реализовать аналогичные сценарии `change-delay`, `jitter-delay`, `correlation-delay` и `distribution-delay`.
5. Сохранить исходные скрипты и результаты измерений, подтвердить корректность настройками, выводом команд и графиками.

Теоретическое введение

Mininet создаёт виртуальные хосты, коммутаторы и сетевые соединения на одном Linux-узле. **NETEM** — дисциплина очередей Linux, управляемая утилитой `tc`; она вносит заданные задержки и случайные вариации на выходе сетевого интерфейса.

RTT (Round-Trip Time) — время от отправки ICMP-запроса до получения ответа. Поскольку NETEM действует на исходящий интерфейс, задержка 100 мс на `h1-eth0` увеличивает RTT приблизительно на 100 мс; при дополнительной задержке 100 мс на `h2-eth0` ожидается приблизительно 200 мс. Реальное RTT включает накладные расходы виртуальной среды.

Джиттер — изменение задержки между пакетами. В записи `delay 100ms 10ms` первый параметр задаёт базовую задержку, второй — вариацию. Дополнительный `25%` задаёт корреляцию последовательных случайных значений, а `distribution normal` выбирает нормальный закон их формирования.

Для статистической оценки использованы: минимум, среднее, максимум и стандартное отклонение RTT. В интерактивной части приведено поле `mdev` непосредственно из `ping`, а в автоматической части стандартное отклонение вычислено программой Python с `statistics.pstdev` по значениям `ping.dat`.

Топология экспериментов:

```
h1 (10.0.0.1) ---- s1 ---- h2 (10.0.0.2)
       NETEM                NETEM (только в двусторонних опытах)
```

Выполнение лабораторной работы

1. Создание и проверка топологии

Работа выполнялась в виртуальной машине Mininet через SSH-клиент PuTTY с настроенной переадресацией X11. Топология запускалась командой:

```
sudo mn --topo=single,2 -x
```

Команда создала хосты `h1`, `h2`, коммутатор `s1` и контроллер `c0`. Командой `ifconfig` проверены интерфейсы `h1-eth0` (`10.0.0.1/8`) и `h2-eth0` (`10.0.0.2/8`).

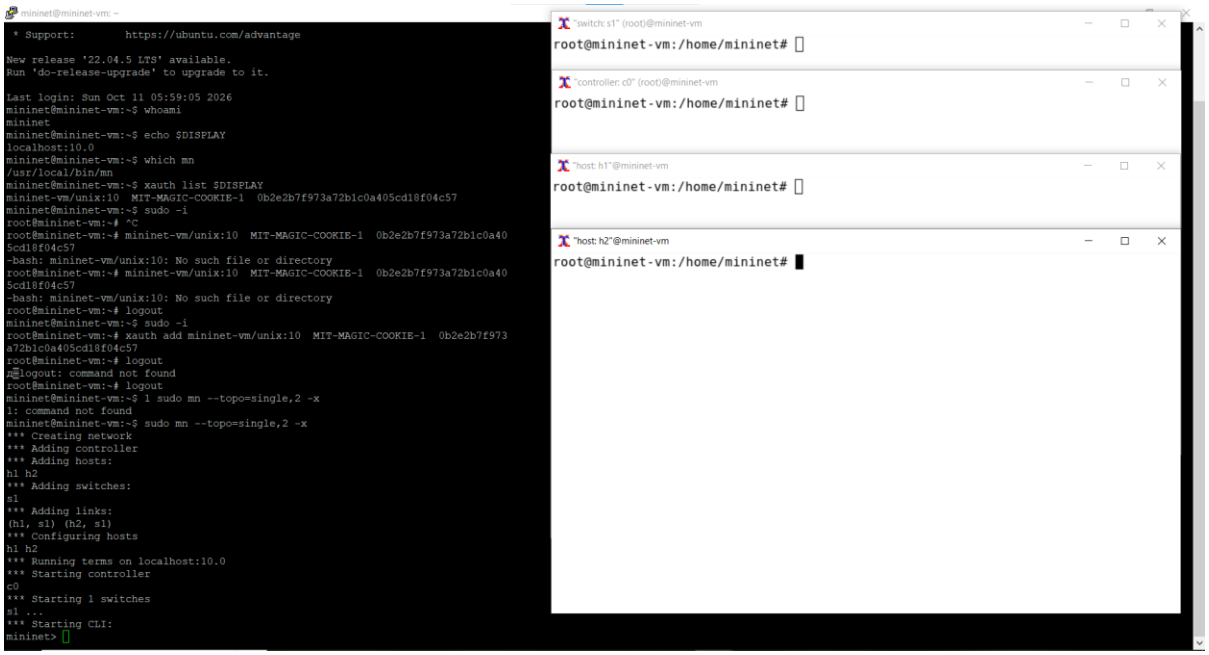


Рисунок 1: Создание виртуальной топологии Mininet и графических терминалов

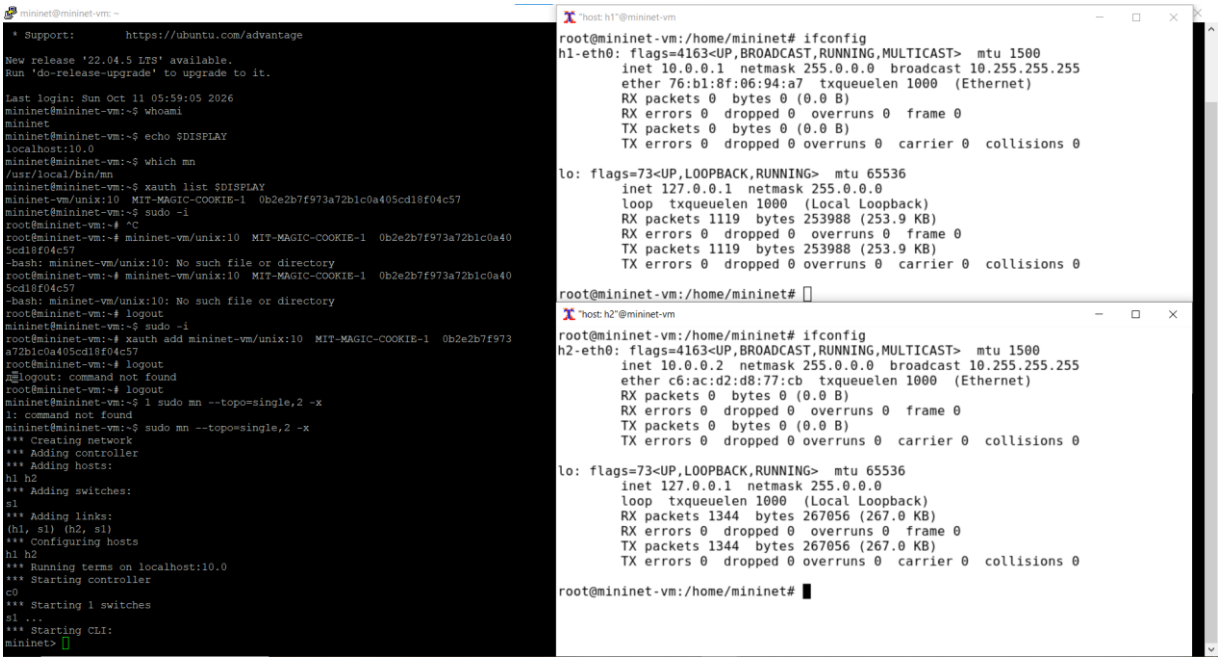


Рисунок 2: Параметры сетевых интерфейсов виртуальных хостов h1 и h2

Базовая проверка связи выполнена на h1:

```
ping -c 6 10.0.0.2
```

```
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.07 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.269 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.097 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.069 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.073 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5093ms
rtt min/avg/max/mdev = 0.069/0.442/2.071/0.731 ms
root@mininet-vm:/home/mininet#
```

Рисунок 3: Базовая проверка связности хостов: 6 ответов и 0 % потерь

Измерено: `min=0,069`, `avg=0,442`, `max=2,071`, `mdev=0,731` мс. Потерь пакетов нет.

2. Интерактивные эксперименты NETEM

2.1. Постоянная задержка 100 мс на одном и двух хостах

На исходящем интерфейсе первого хоста введена задержка:

```
sudo tc qdisc add dev h1-eth0 root netem delay 100ms
ping -c 6 10.0.0.2
```

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 100.305/102.922/107.715/3.166 ms
root@mininet-vm:/home/mininet#
```

Рисунок 4: RTT после добавления 100 мс только на h1

Средний RTT вырос до **102,922 мс**, что близко к заданной односторонней задержке.

Затем на `h2-eth0` также установлена задержка 100 мс:

```
sudo tc qdisc add dev h2-eth0 root netem delay 100ms
```

```
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=203 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=218 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=202 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=202 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=202 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=207 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5252ms
rtt min/avg/max/mdev = 201.560/205.462/217.713/5.815 ms
root@mininet-vm:/home/mininet#
```

Рисунок 5: Двусторонняя задержка: RTT в среднем 205,462 мс

В двустороннем опыте измерено `min=201,560`, `avg=205,462`, `max=217,713`, `mdev=5,815` мс. Таким образом, две выходные задержки по 100 мс суммируются в RTT.

2.2. Изменение задержки до 50 мс

Действующие правила изменены на обоих интерфейсах:

```
sudo tc qdisc change dev h1-eth0 root netem delay 50ms
sudo tc qdisc change dev h2-eth0 root netem delay 50ms
ping -c 6 10.0.0.2
```

```
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h1-eth0 root netem dela
y 50ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=115 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=111 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=138 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=112 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=112 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5030ms
rtt min/avg/max/mdev = 101.049/114.759/138.253/11.332 ms
root@mininet-vm:/home/mininet#
```

Рисунок 6: Измерение RTT при задержке 50 мс на каждом хосте

Средний RTT составил **114,759 мс**, а минимальный — **101,049 мс**. Значения близки к ожидаемым 100 мс с учётом накладных расходов.

2.3. Удаление правил

После выполнения `tc qdisc del` на обоих интерфейсах базовая задержка восстановилась:

```
sudo tc qdisc del dev h1-eth0 root netem
sudo tc qdisc del dev h2-eth0 root netem
ping -c 6 10.0.0.2
```

```
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.814 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.695 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.146 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.088 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.077 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5381ms
rtt min/avg/max/mdev = 0.072/0.315/0.814/0.313 ms
root@mininet-vm:/home/mininet#
```

Рисунок 7: Проверка сети после удаления правил NETEM

Получено `min=0,072`, `avg=0,315`, `max=0,814`, `mdev=0,313` мс, потери 0 %.

2.4. Джиттер, корреляция и распределение задержки

На h1 последовательно выполнены три независимых сценария. Перед установкой нового режима предыдущая дисциплина NETEM удалялась.

```
# Джиттер
sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms
ping -c 6 10.0.0.2

# После очистки: корреляция 25 %
sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms 25%
ping -c 20 10.0.0.2

# После очистки: нормальное распределение
sudo tc qdisc add dev h1-eth0 root netem delay 100ms 20ms distribution normal
ping -c 10 10.0.0.2
```

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 10ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=92.8 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=93.7 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=93.5 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=99.1 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5033ms
rtt min/avg/max/mdev = 92.805/98.608/110.019/6.187 ms
root@mininet-vm:/home/mininet#
```

Рисунок 8: Интерактивное измерение RTT при джиттере 10 мс

```
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 10ms 25%
root@mininet-vm:/home/mininet# ping -c 20 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=93.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=118 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=98.9 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=92.5 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=92.7 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=181 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=93.6 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=185 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=126 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=119 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=115 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=117 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=108 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19403ms
rtt min/avg/max/mdev = 92.524/114.183/184.689/24.646 ms
root@mininet-vm:/home/mininet#
```

Рисунок 9: Интерактивное измерение при корреляции 25 %

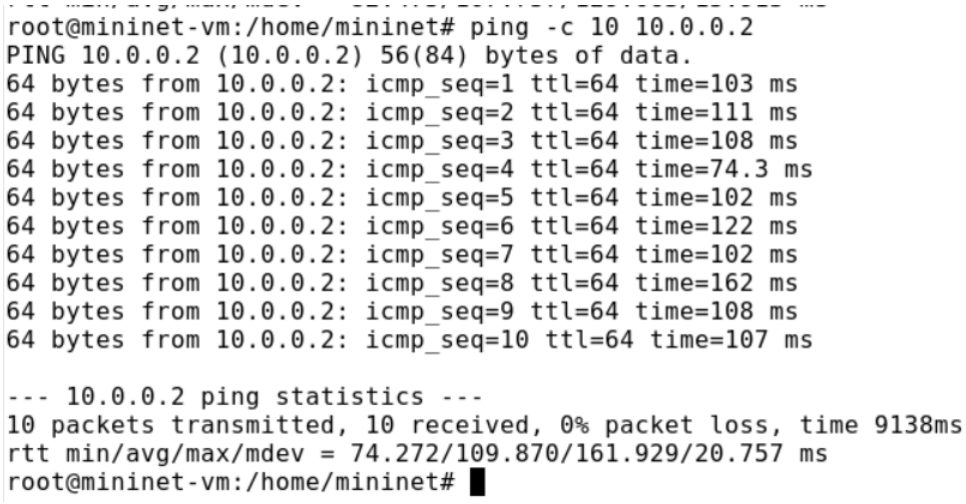


Рисунок 10: Интерактивное измерение при нормальном распределении задержки

Сводная таблица интерактивных экспериментов

Сценарий	ICMP-ответов	Min, мс	Avg, мс	Max, мс	mdev, мс
Без NETEM	6	0,069	0,442	2,071	0,731
100 мс на h1	6	100,305	102,922	107,715	3,166
100 мс на h1 и h2	6	201,560	205,462	217,713	5,815
50 мс на h1 и h2	6	101,049	114,759	138,253	11,332
После удаления NETEM	6	0,072	0,315	0,814	0,313
Джиттер 100 ± 10 мс	6	92,805	98,608	110,019	6,187
Корреляция 25 %	20	92,524	114,183	184,689	24,646
Нормальное распределение	10	74,272	109,870	161,929	20,757

Все показанные серии закончились без потерь ICMP-пакетов. Виртуальная машина может вносить дополнительные задержки, поэтому среднее не всегда совпадает с заданным параметром NETEM.

3. Воспроизводимый эксперимент `simple-delay`

3.1. Подготовка окружения и проекта

Обновлены списки пакетов и установлен просмотрщик Geeqie. Для проекта создан каталог `~/work/lab_netem_i/simple-delay` с файлами `lab_netem_i.py`, `ping_plot`, `Makefile` и позднее `ping_stats.py`.

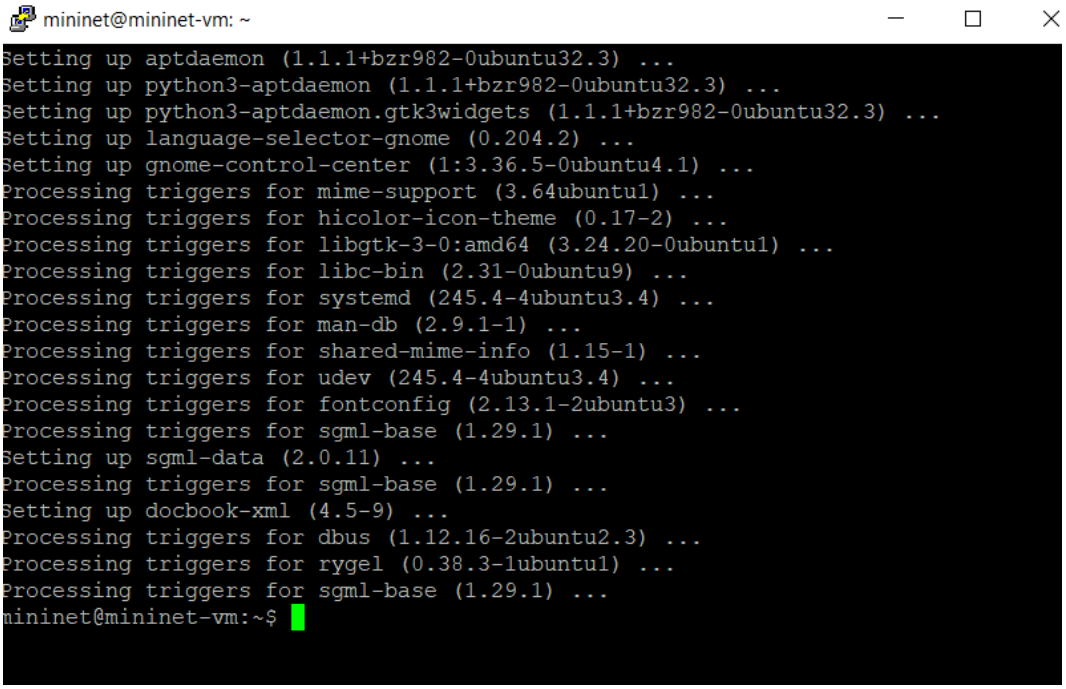


Рисунок 11: Завершение установки графических зависимостей

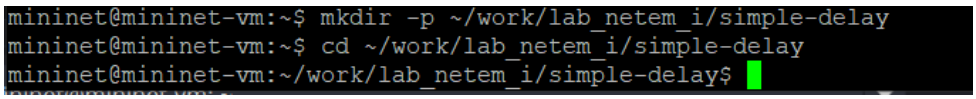


Рисунок 12: Создание каталога первого воспроизводимого эксперимента

3.2. Программа эксперимента

В Python API Mininet создаются `h1`, `h2`, `s1` и `c0`, затем на обоих исходящих интерфейсах устанавливается по 100 мс задержки. После 10-секундной паузы хост `h1` выполняет 100 ICMP-запросов к `h2`. Конвейер `grep` / `awk` / `sed` извлекает порядковый номер запроса и RTT, формируя двухколоночный файл `ping.dat`.

Ключевой фрагмент `lab_netem_i.py`:

```
h1.cmdPrint('tc qdisc add dev h1-eth0 root netem delay 100ms')
h2.cmdPrint('tc qdisc add dev h2-eth0 root netem delay 100ms')
time.sleep(10)
h1.cmdPrint(
    'ping -c 100', h2.IP(),
    '| grep "time=" | awk \'{print $5, $7}\' | '
    'sed -e \'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat'
)
```

Полная программа хранится в `simple-delay/lab_netem_i.py`.

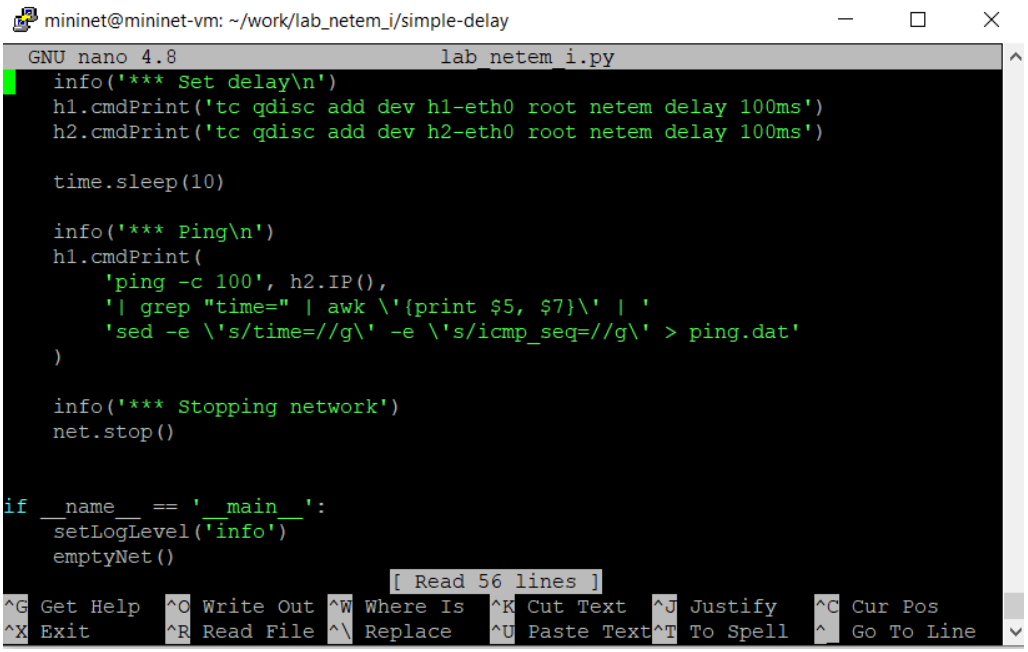


Рисунок 13: Импорты Python и создание виртуальной сети в lab_netem_i.py

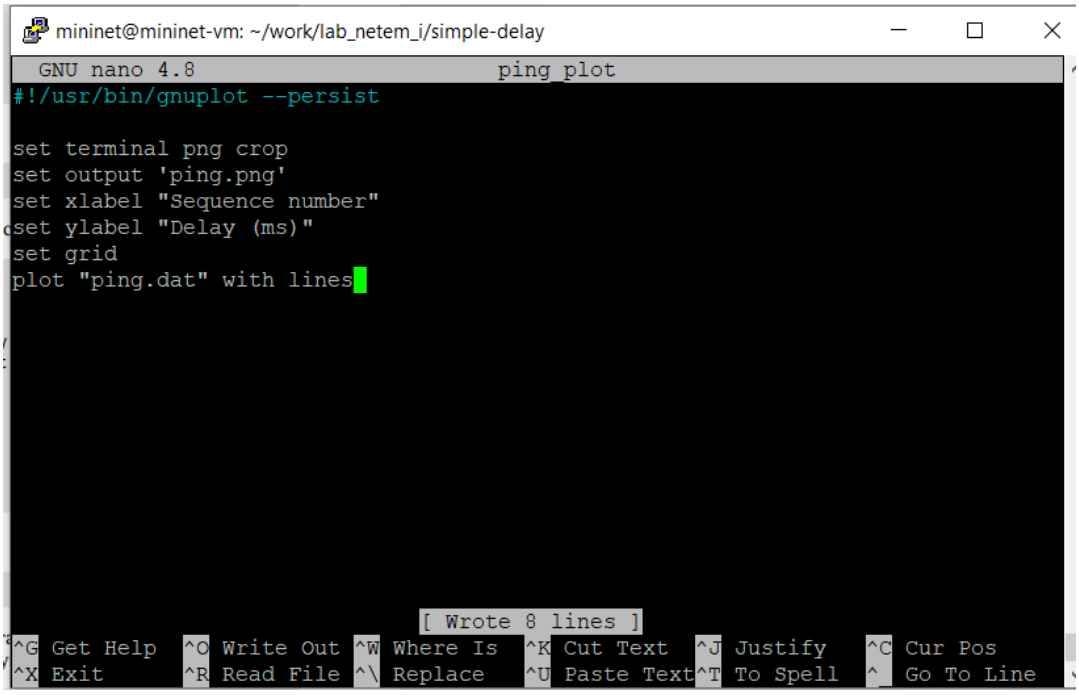


Рисунок 14: Настройка NETEM и формирование данных ping.dat

3.3. Визуализация и автоматизация

Для Gnuplot создан скрипт `ping_plot`. Makefile запускает Python-программу, корректирует владельца данных, строит график, а цель `clean` удаляет полученные `.dat` и `.png`-файлы.

```
all: ping.dat ping.png stats

ping.dat:
    sudo python lab_netem_i.py
```

```
    sudo chown mininet:mininet ping.dat

ping.png: ping.dat
    ./ping_plot

stats: ping.dat
    python3 ping_stats.py

clean:
    -rm -f *.dat *.png
```

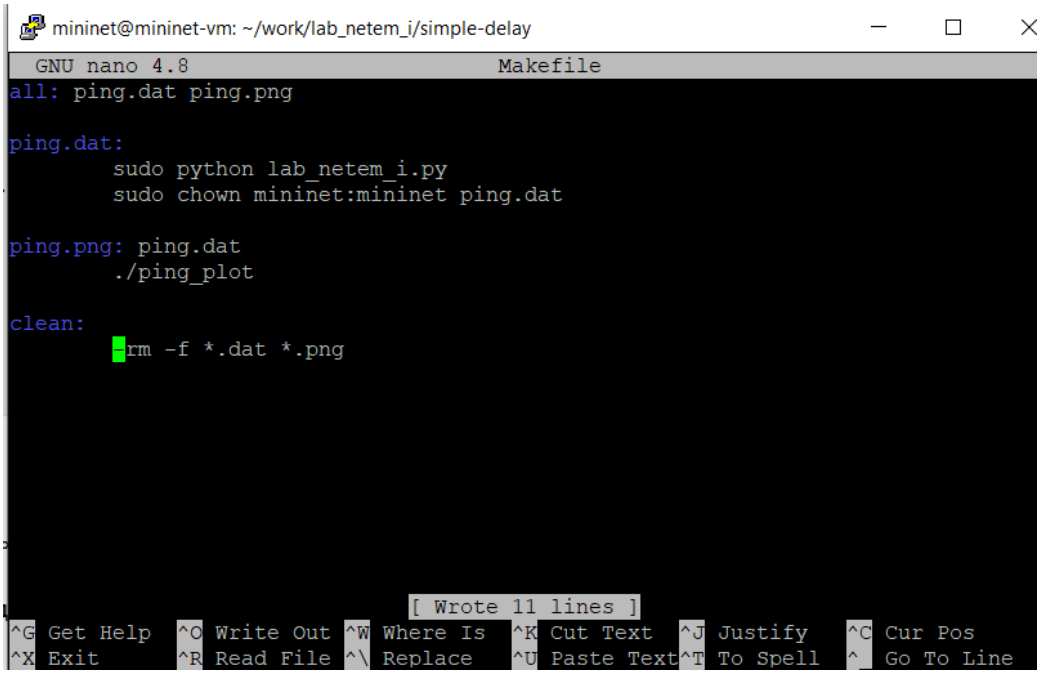


Рисунок 15: Скрипт Gnuplot для построения графика ping.png

```
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk \'{print $5, $7}\' |
sed -e \'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
./ping_plot
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$
```

Рисунок 16: Первоначальный Makefile для автоматического запуска

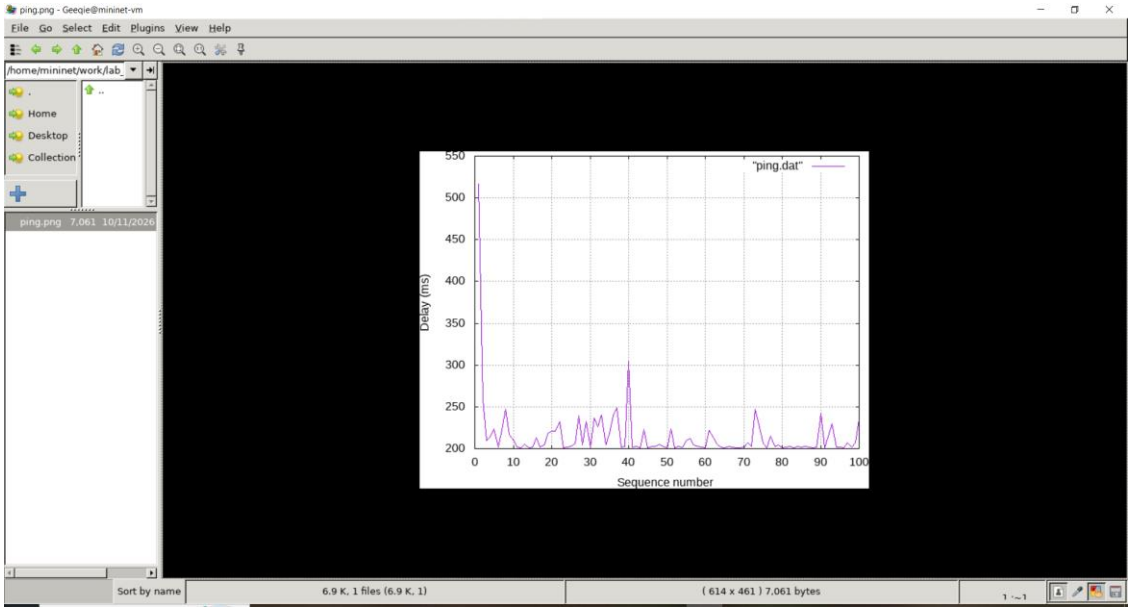


Рисунок 17: Успешный запуск make и построение файла ping.png

3.4. Корректировка данных и расчёт статистики

Первоначальный график содержит заметный начальный выброс. По методичке первая строка `ping.dat` была удалена, после чего построен обновлённый график. Далее реализован расчёт минимума, среднего, максимума и стандартного отклонения RTT. Для этого первого набора после удаления одной строки осталось **99** значений.

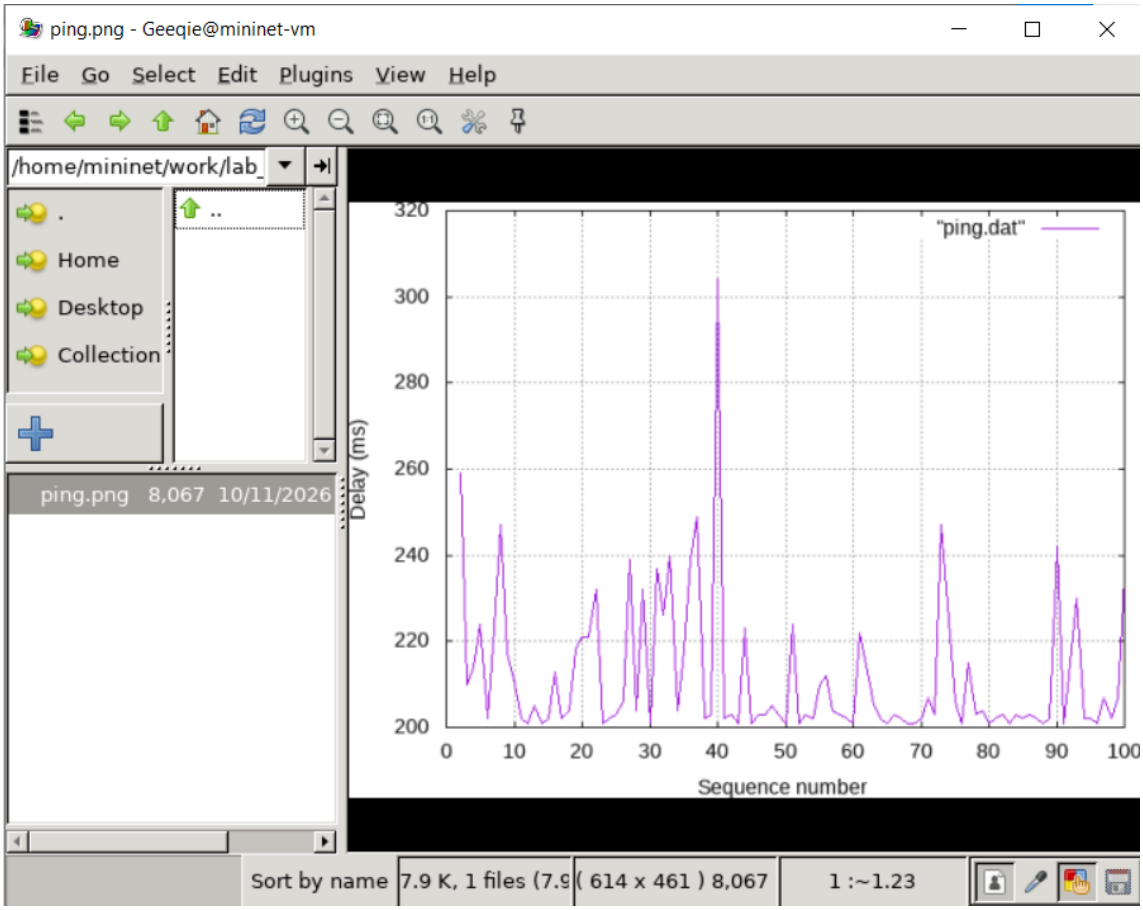


Рисунок 18: Первоначальный график RTT из 100 пакетов

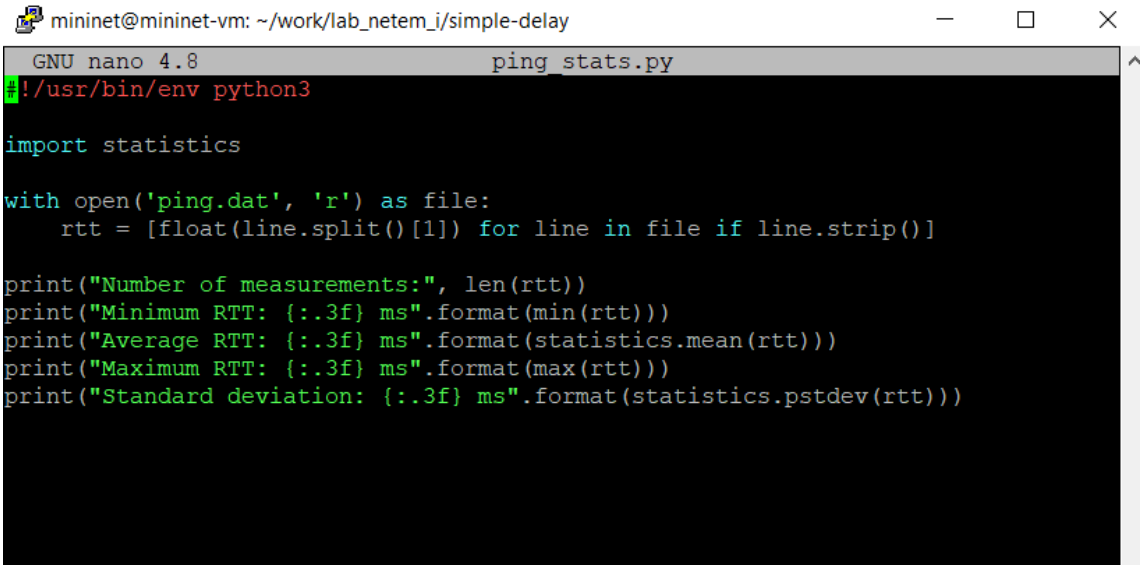


Рисунок 19: График после удаления первого измерения

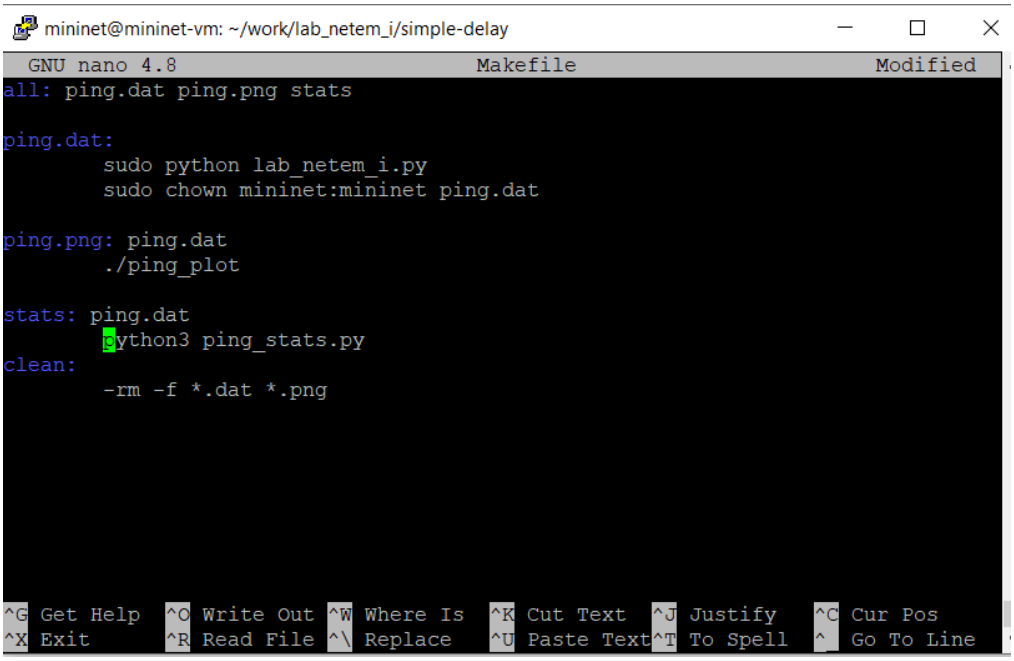


Рисунок 20: Makefile с добавленной целью stats

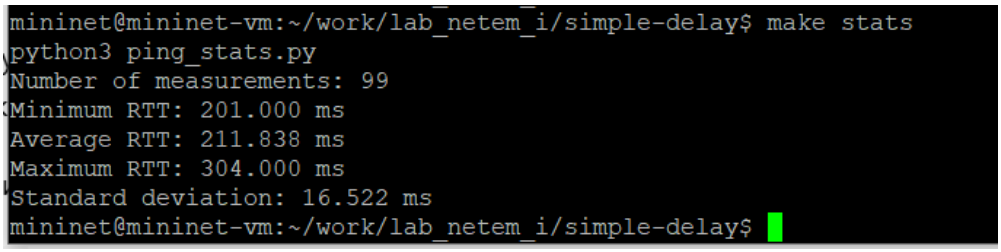


Рисунок 21: Расчёт статистики по 99 значениям RTT

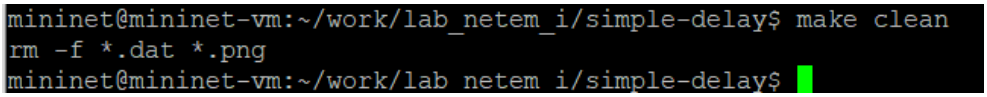


Рисунок 22: Удаление данных и графика командой make clean

Изменённый набор из 99 наблюдений: `min=201,000`, `avg=211,838`, `max=304,000`, `σ=16,522` мс. После обязательной очистки эксперимент был повторён, и результаты вновь сохранены (см. раздел 7).

4. Самостоятельный эксперимент `change-delay`

Сценарий состоял из двух последовательных серий по 100 измерений. Сначала на обоих хостах устанавливалась задержка 100 мс, затем она изменялась командой `tc qdisc change` до 50 мс. Данные сохранялись отдельно в `ping_100.dat` и `ping_50.dat`. Скрипт Gnuplot построил сравнительный график из двух рядов.

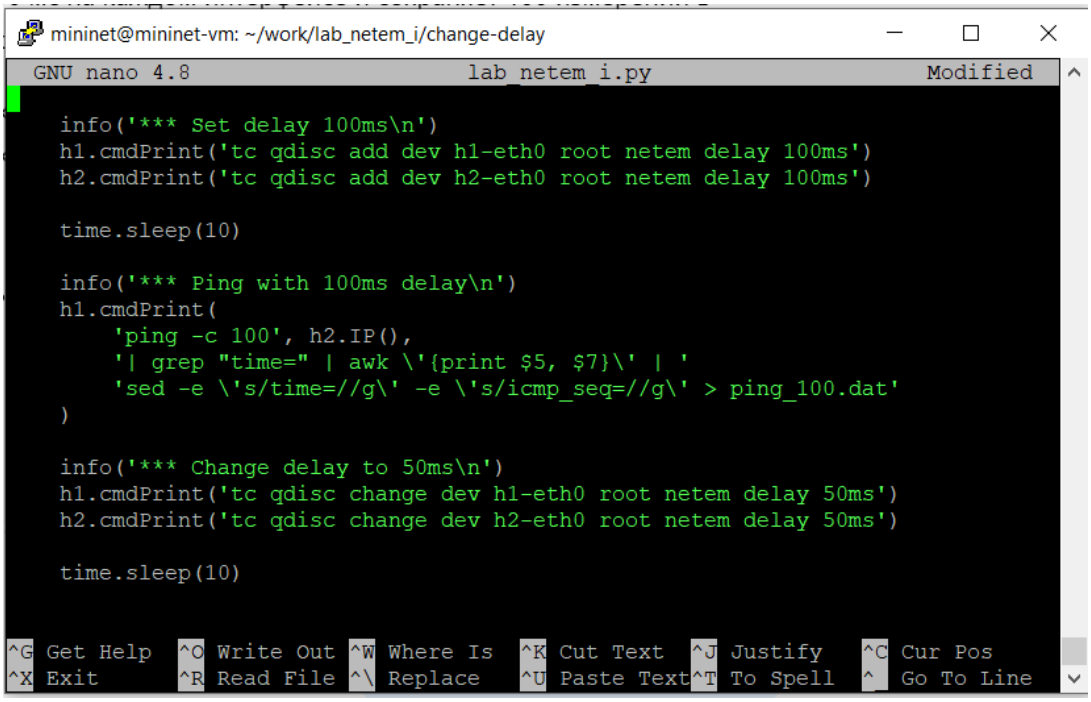


Рисунок 23: Первая фаза программы change-delay: задержка 100 мс

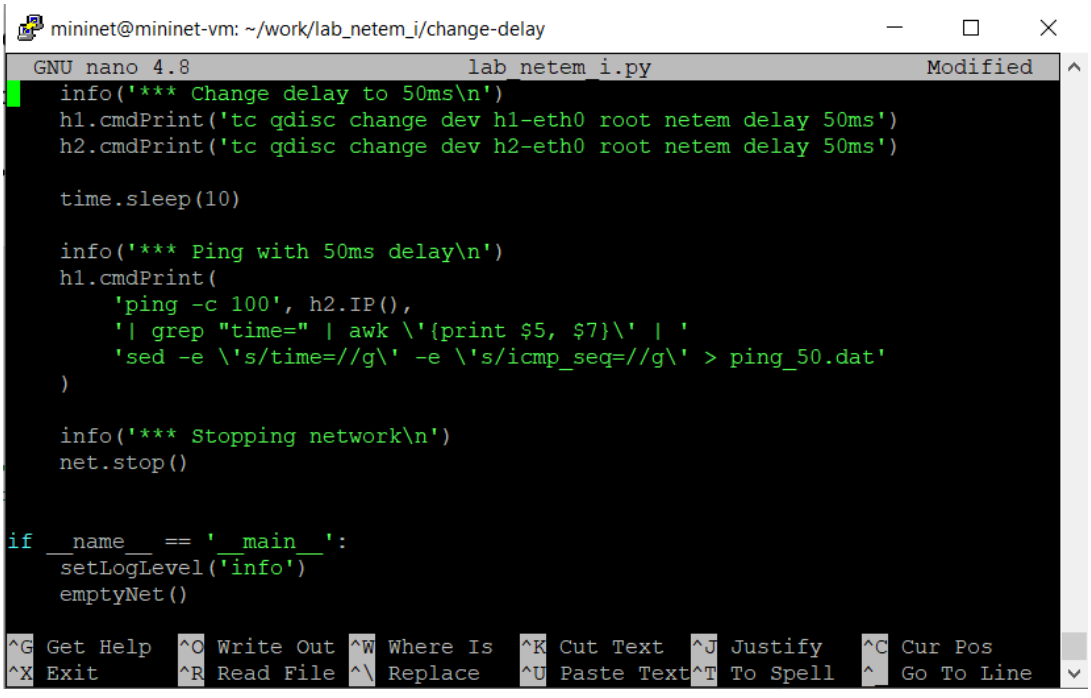


Рисунок 24: Первая фаза программы change-delay: задержка 100 мс

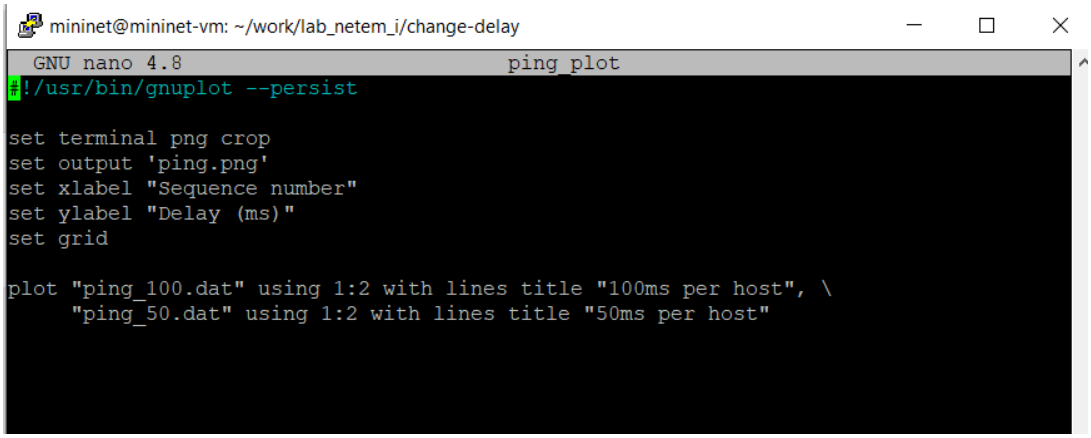


Рисунок 25: Вторая фаза программы change-delay: задержка 50 мс

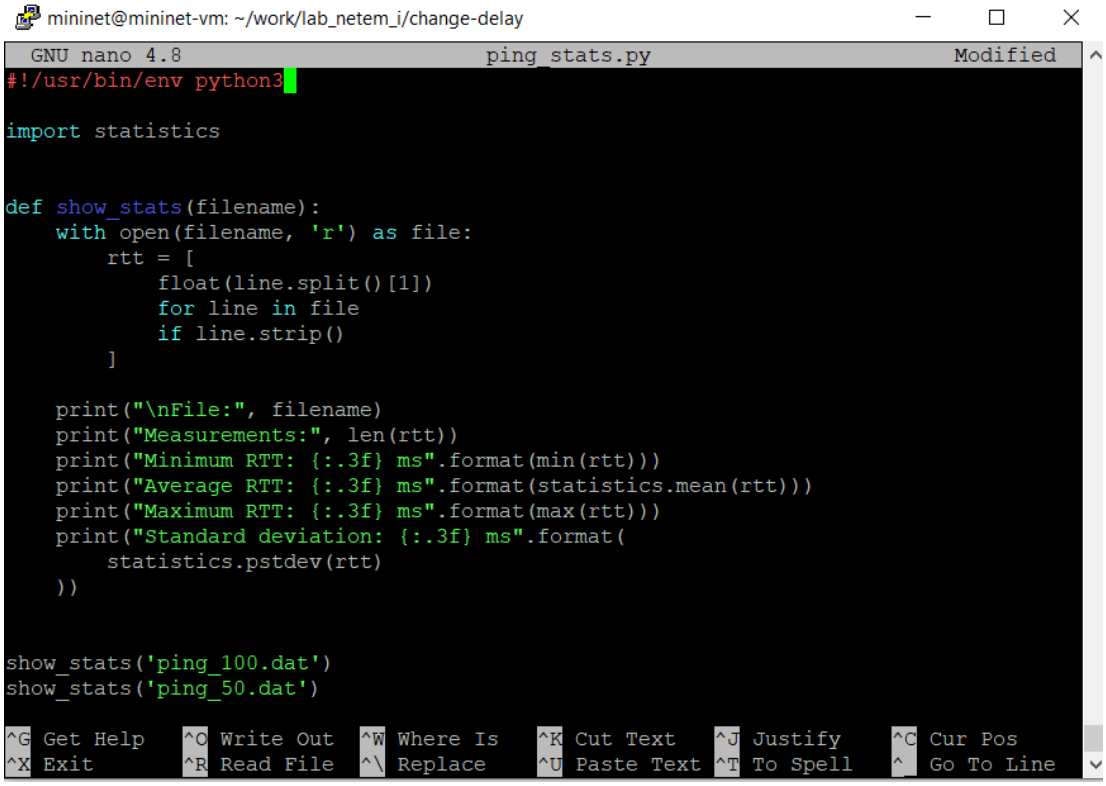


Рисунок 26: Скрипт сравнительного графика для двух наборов данных

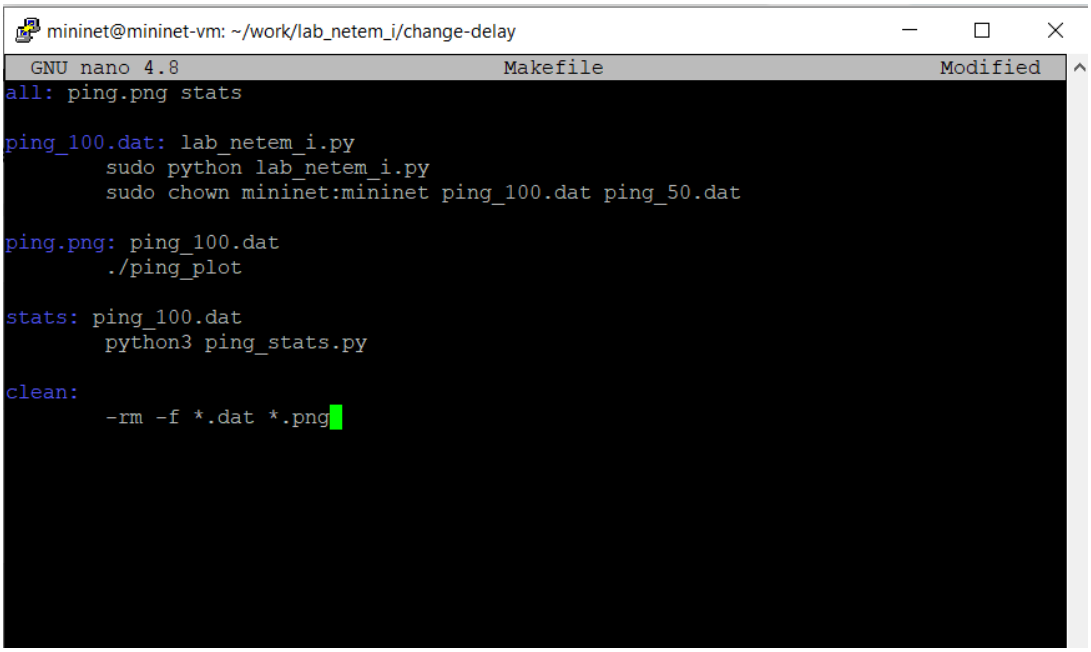


Рисунок 27: Скрипт расчёта статистики двух серий

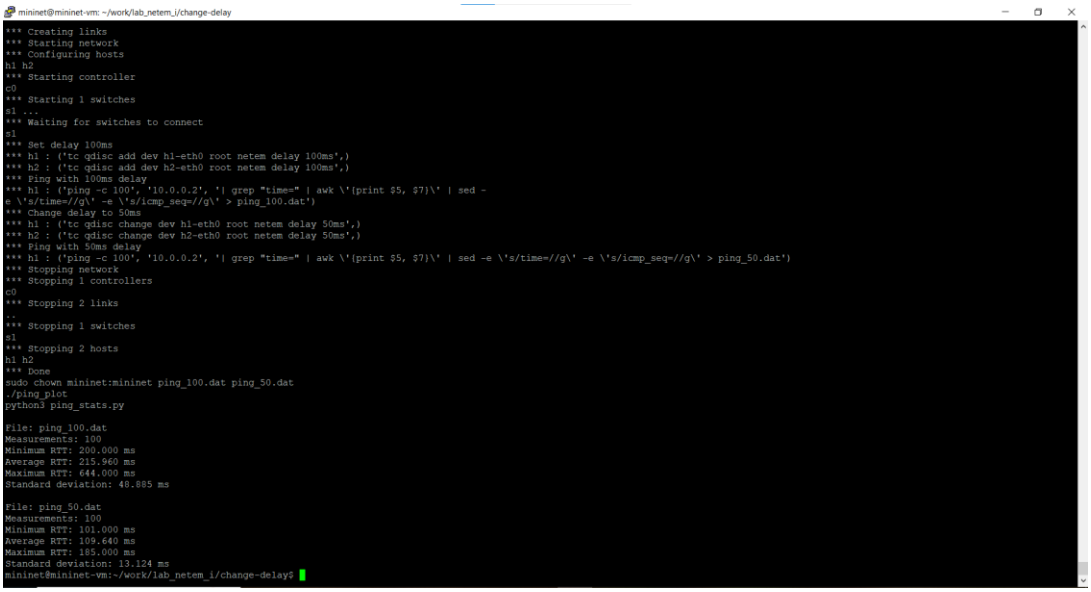


Рисунок 28: Автоматизация двухфазного эксперимента через Makefile

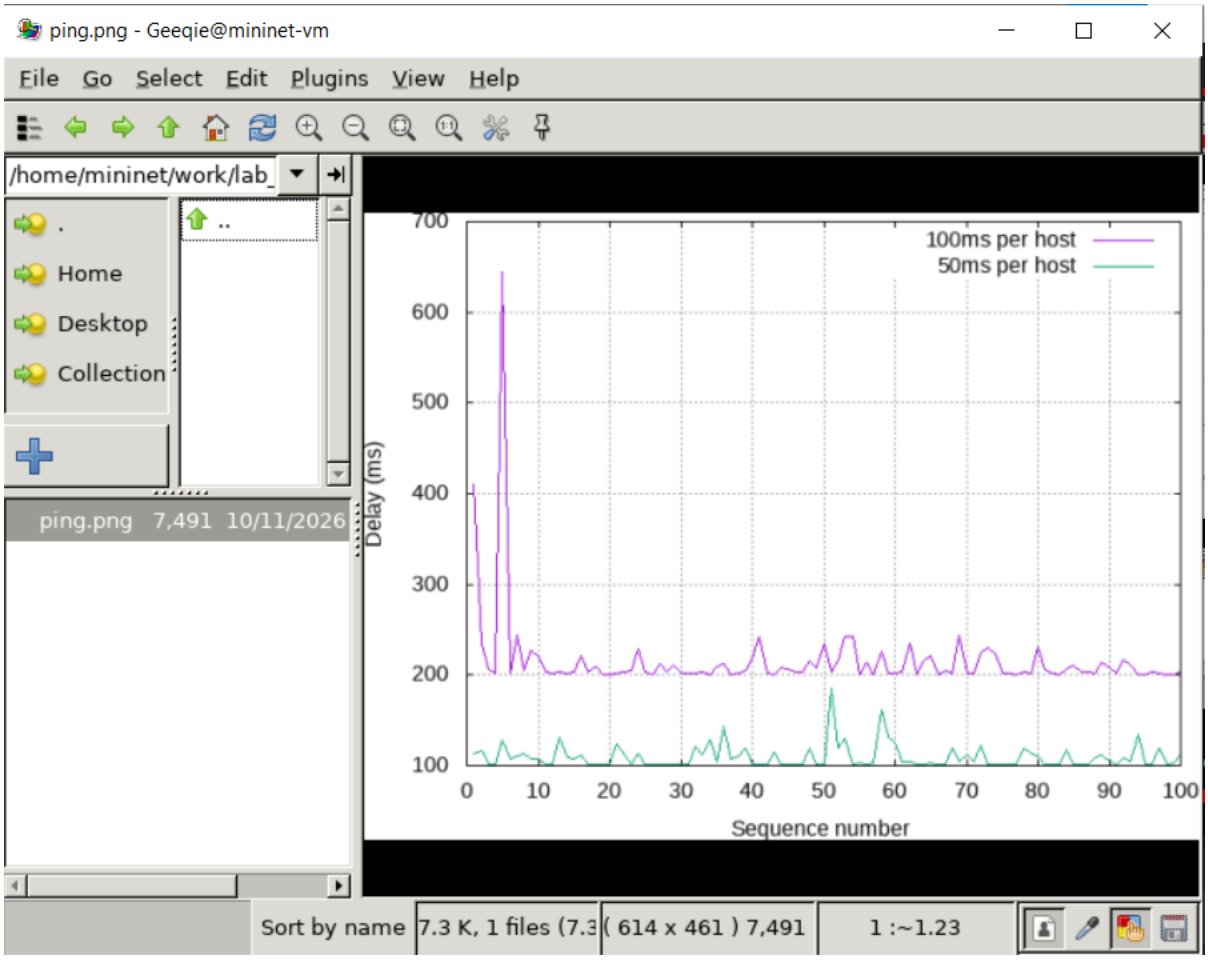


Рисунок 29: Вывод двух серий измерений change-delay

В первом режиме получены `avg=215,960` мс и `σ=48,885` мс, во втором — `avg=109,640` мс и `σ=13,124` мс. Уменьшение настроенной двусторонней задержки приблизительно вдвое сопровождалось аналогичным снижением среднего RTT.

5. Самостоятельный эксперимент `jitter-delay`

В Python-программе на `h1-eth0` применена команда `tc qdisc add dev h1-eth0 root netem delay 100ms 10ms`. Проведено 100 измерений, построен график и рассчитана статистика.

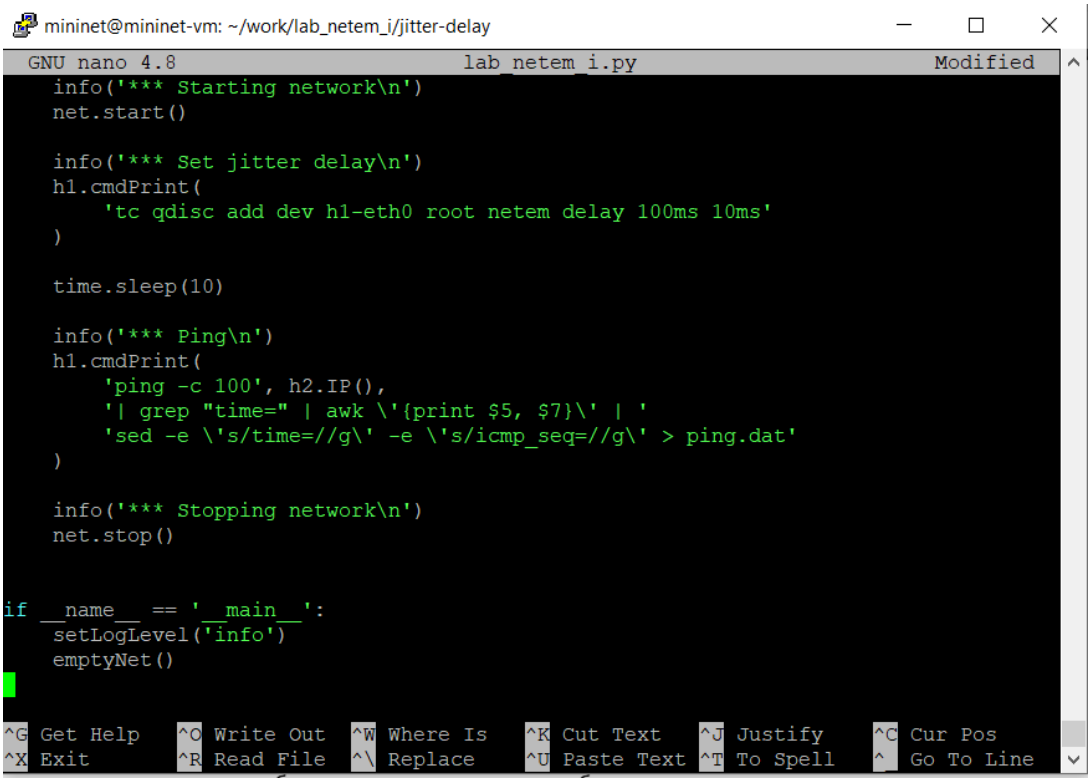


Рисунок 30: Программа воспроизводимого измерения джиттера

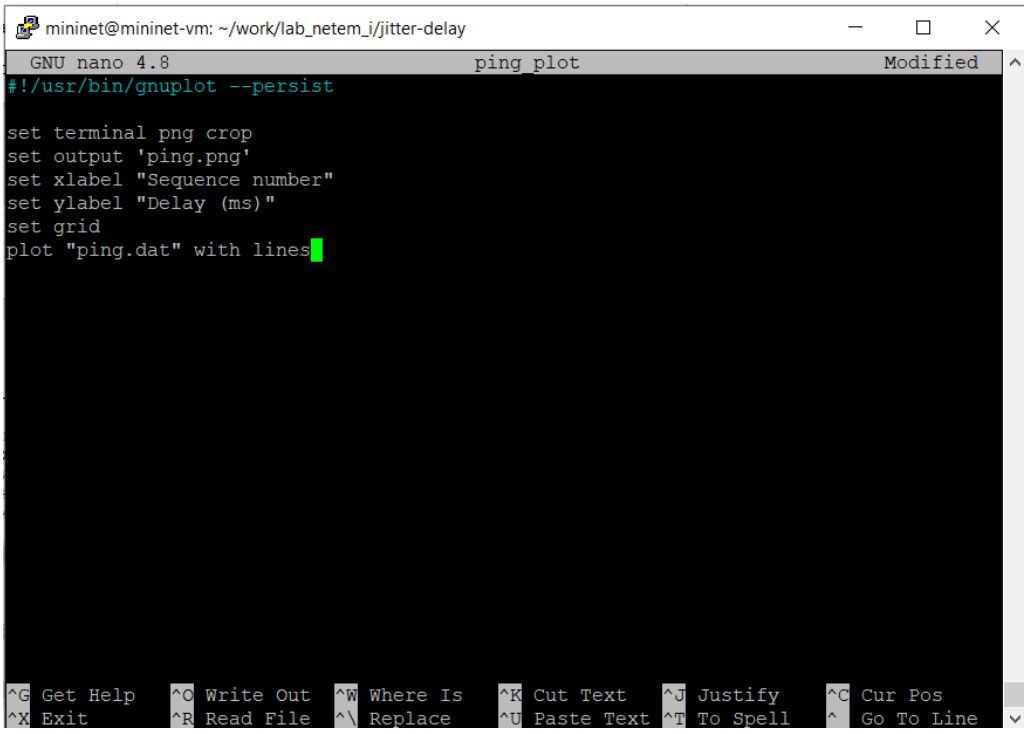


Рисунок 31: Gnuplot-скрипт для графика задержки при джиттере

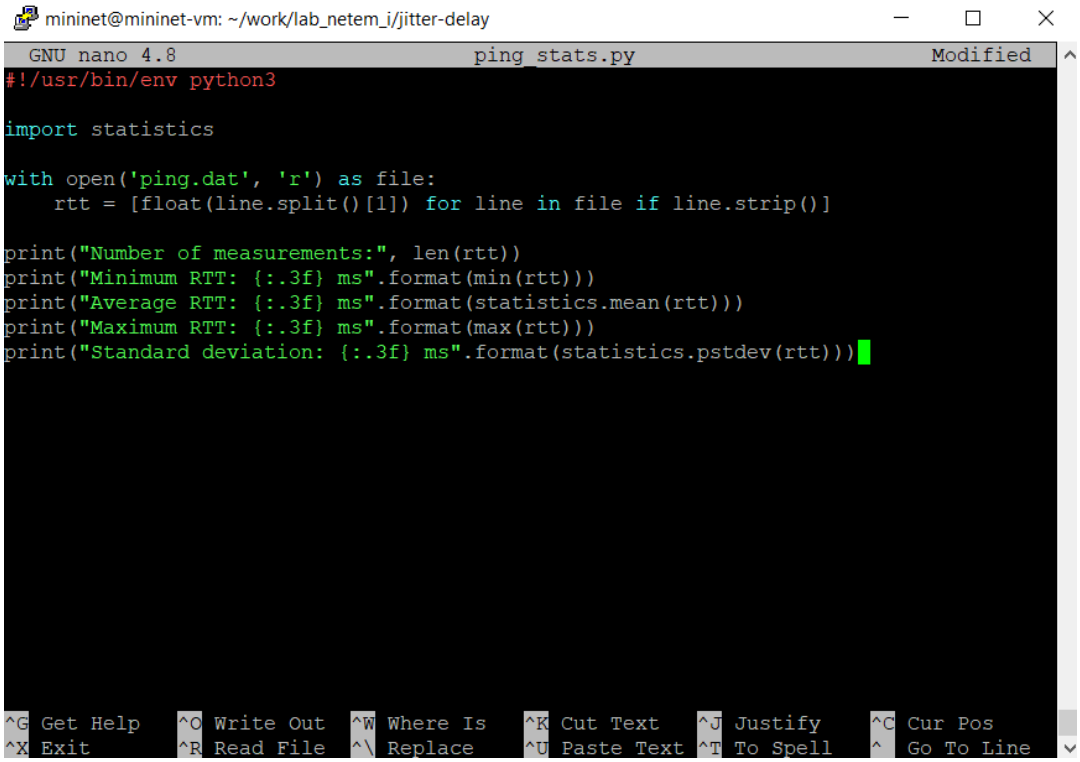


Рисунок 32: Python-скрипт вычисления статистики RTT

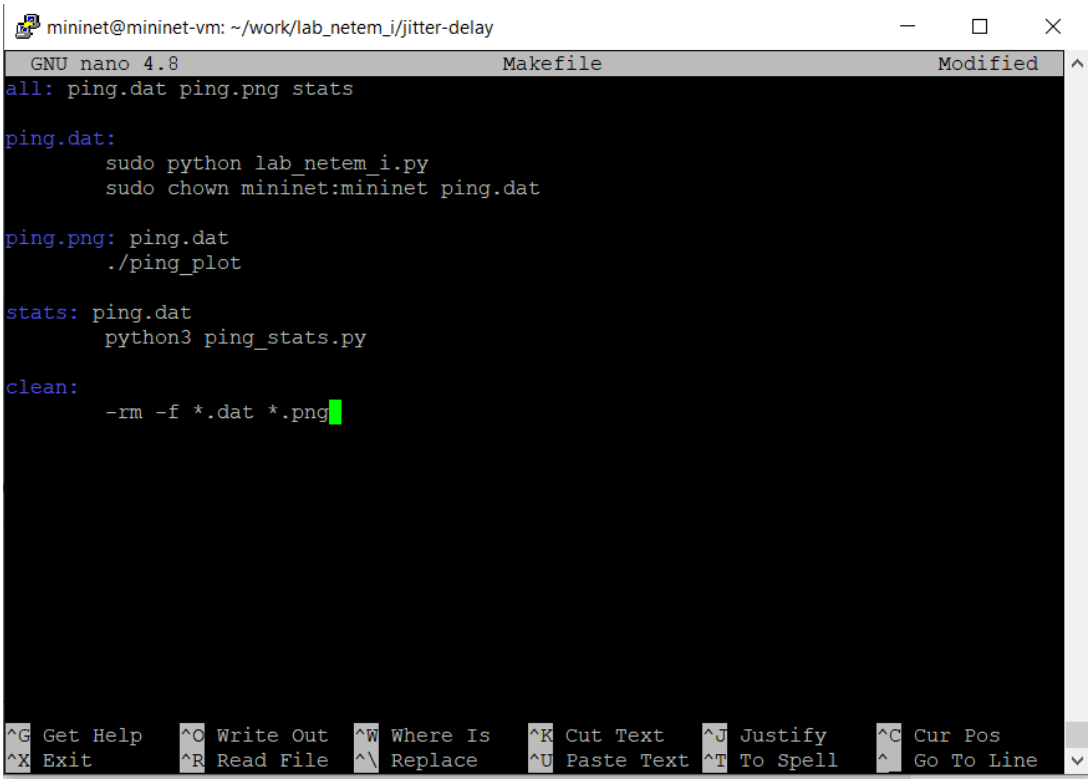


Рисунок 33: Makefile эксперимента jitter-delay

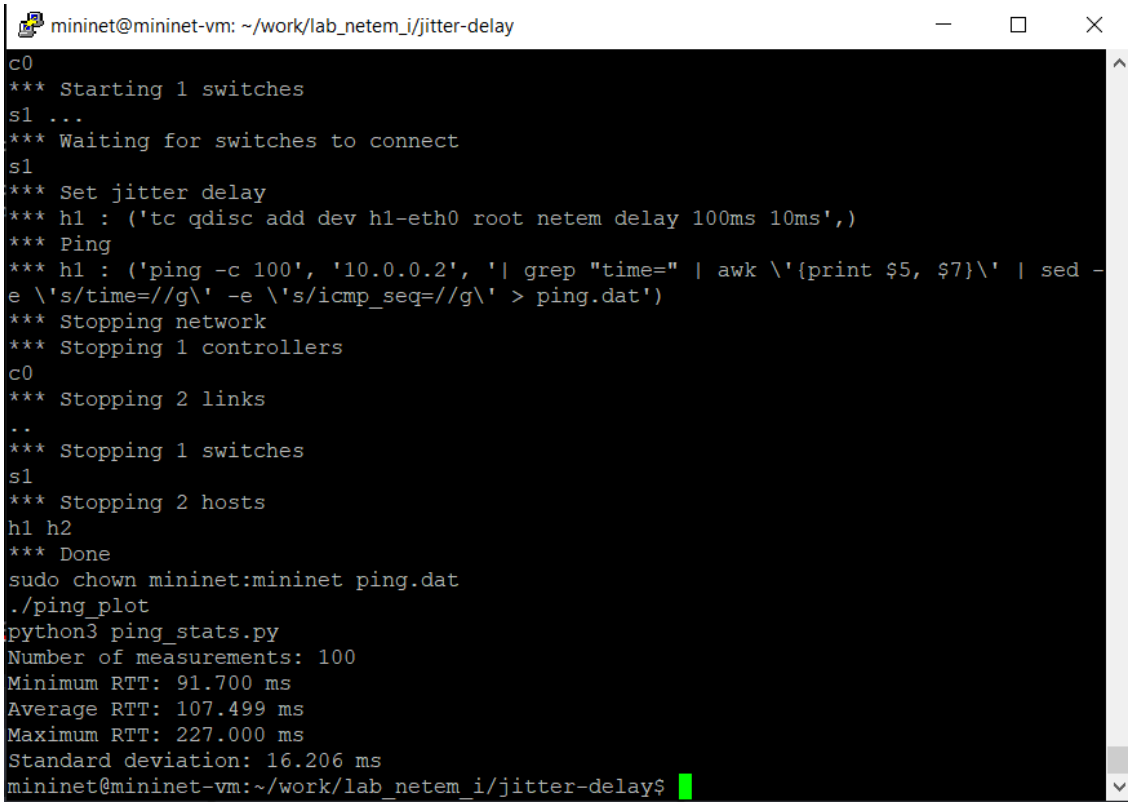


Рисунок 34: Результаты 100 измерений при джиттере

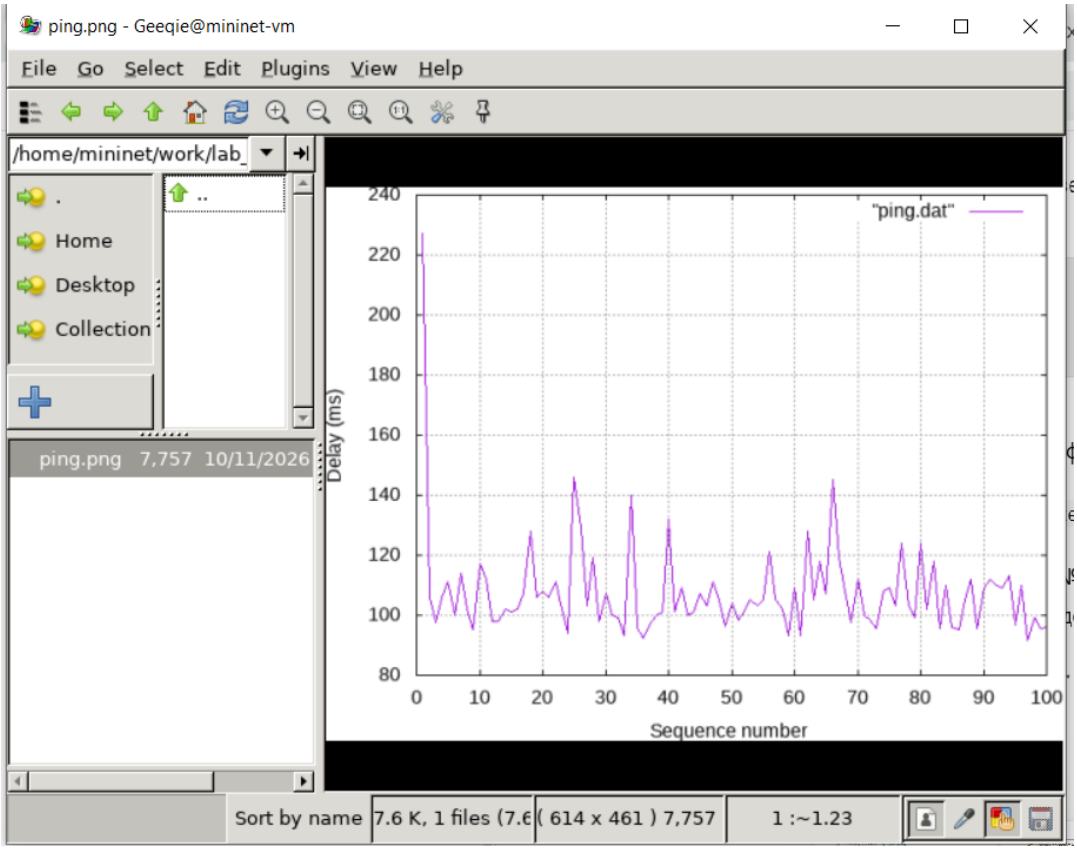


Рисунок 35: График случайных колебаний RTT при джиттере

Измерено `min=91,700`, `avg=107,499`, `max=227,000`, `σ=16,206` мс. Основная масса наблюдений расположена около 100 мс; отдельные выбросы отражают влияние дополнительной задержки во время выполнения.

6. Самостоятельные эксперименты `correlation-delay` и `distribution-delay`

В эксперименте корреляции использовалась настройка `delay 100ms 10ms 25%`. В эксперименте распределения — `delay 100ms 20ms distribution normal`. В обоих случаях NETEM применялся на интерфейсе `h1-eth0`, затем собиралось по 100 наблюдений.

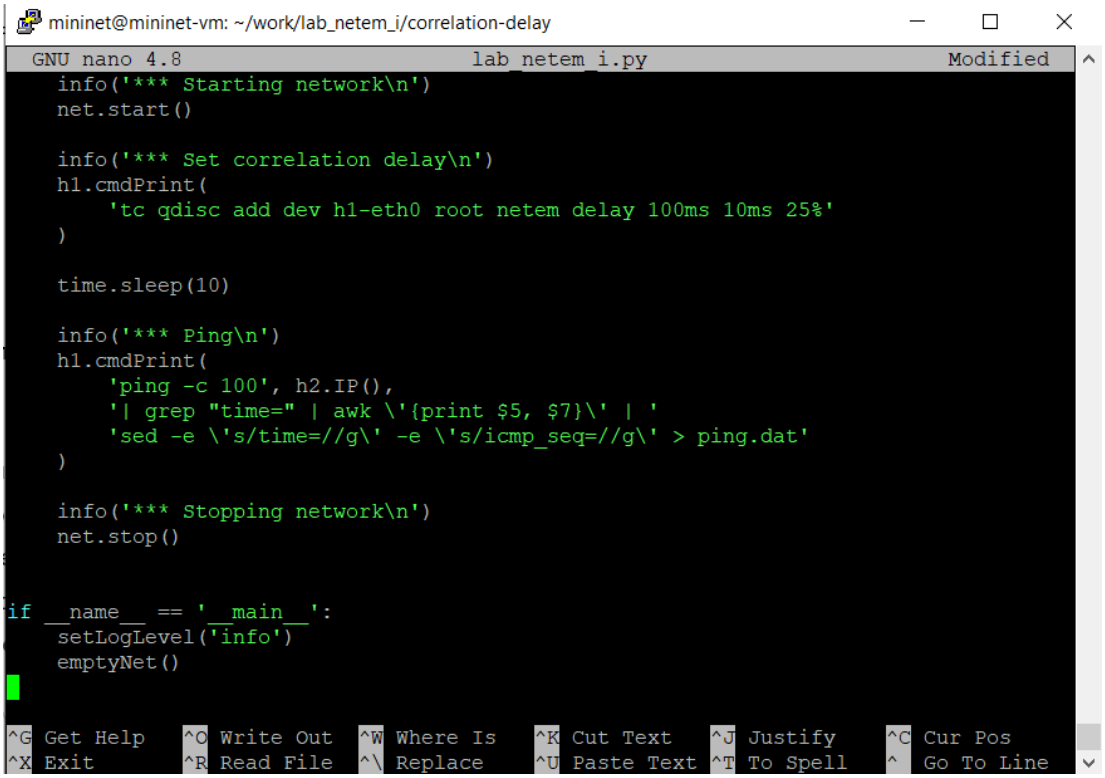


Рисунок 36: Программная настройка корреляции 25 %

```
mininet@mininet-vm: ~/work/lab_netem_i/correlation-delay
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set correlation delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms 10ms 25%',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk \'{print $5, $7}\' | sed -
e \'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat')
*** Stopping network
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
./ping_plot
python3 ping_stats.py
Number of measurements: 100
Minimum RTT: 90.800 ms
Average RTT: 108.485 ms
Maximum RTT: 243.000 ms
Standard deviation: 19.297 ms
mininet@mininet-vm:~/work/lab_netem_i/correlation-delay$
```

Рисунок 37: Результаты измерений с корреляцией 25 %

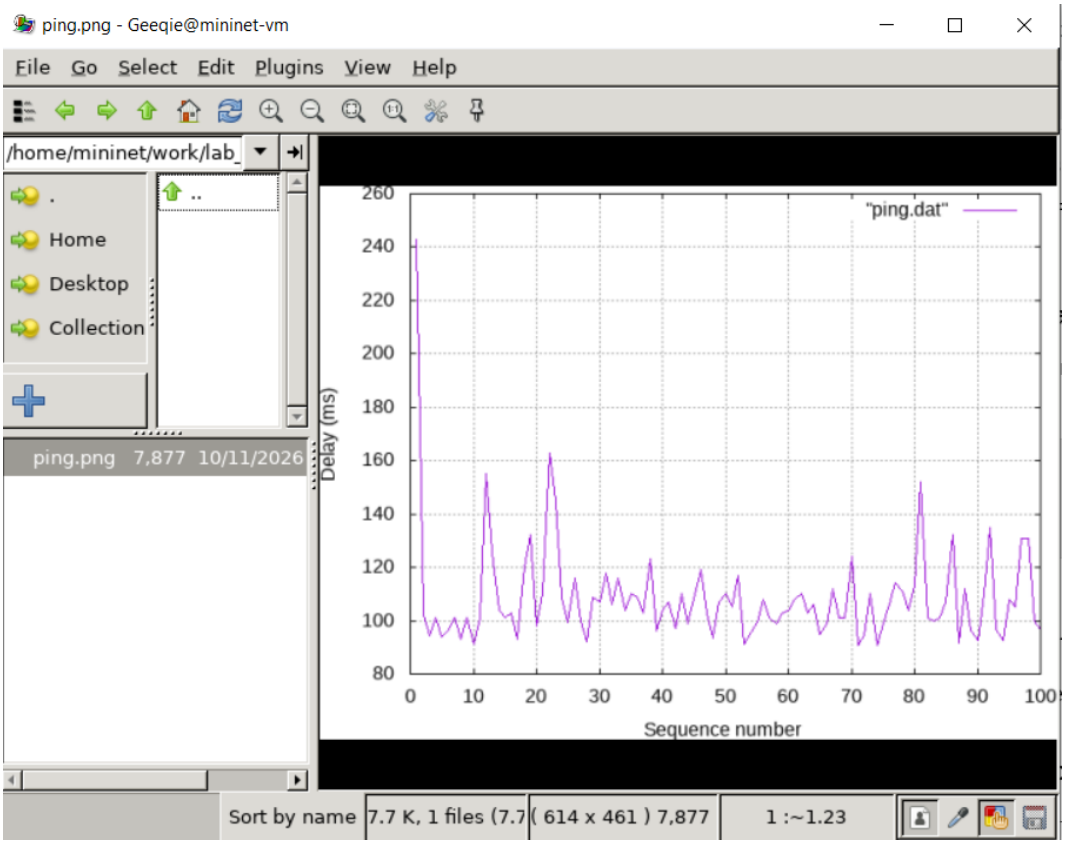


Рисунок 38: График RTT при заданной корреляции

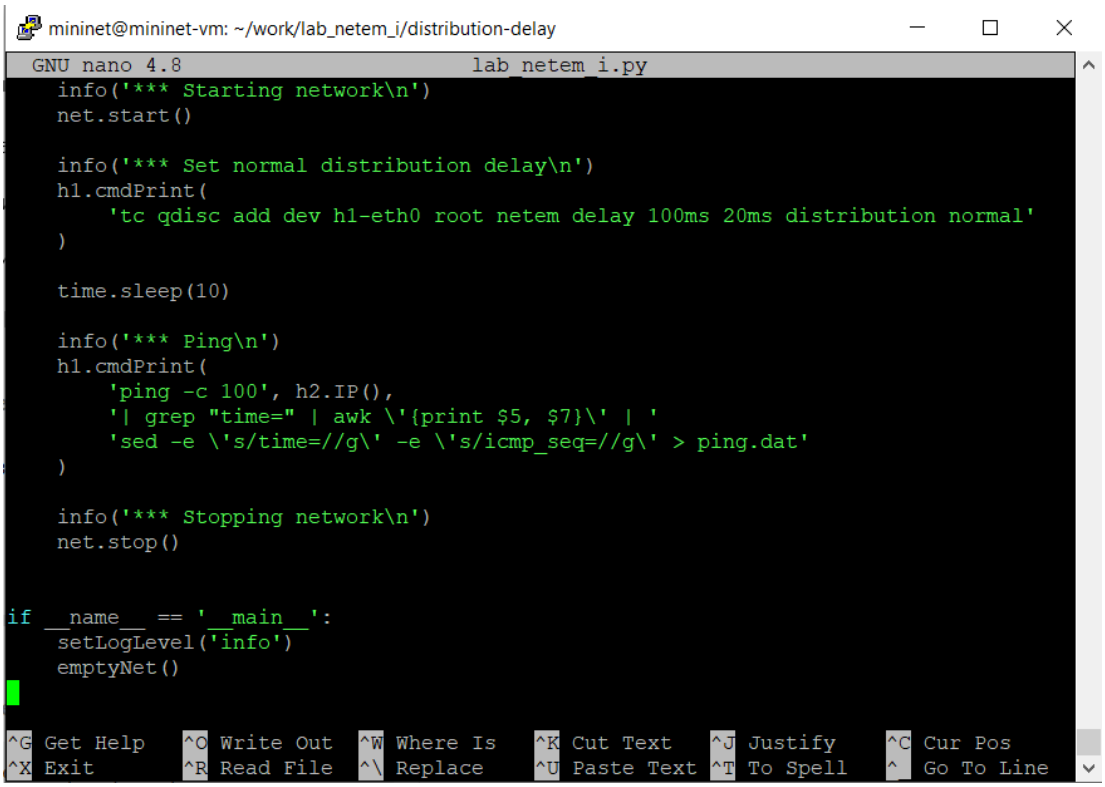


Рисунок 39: Программная настройка нормального распределения задержки

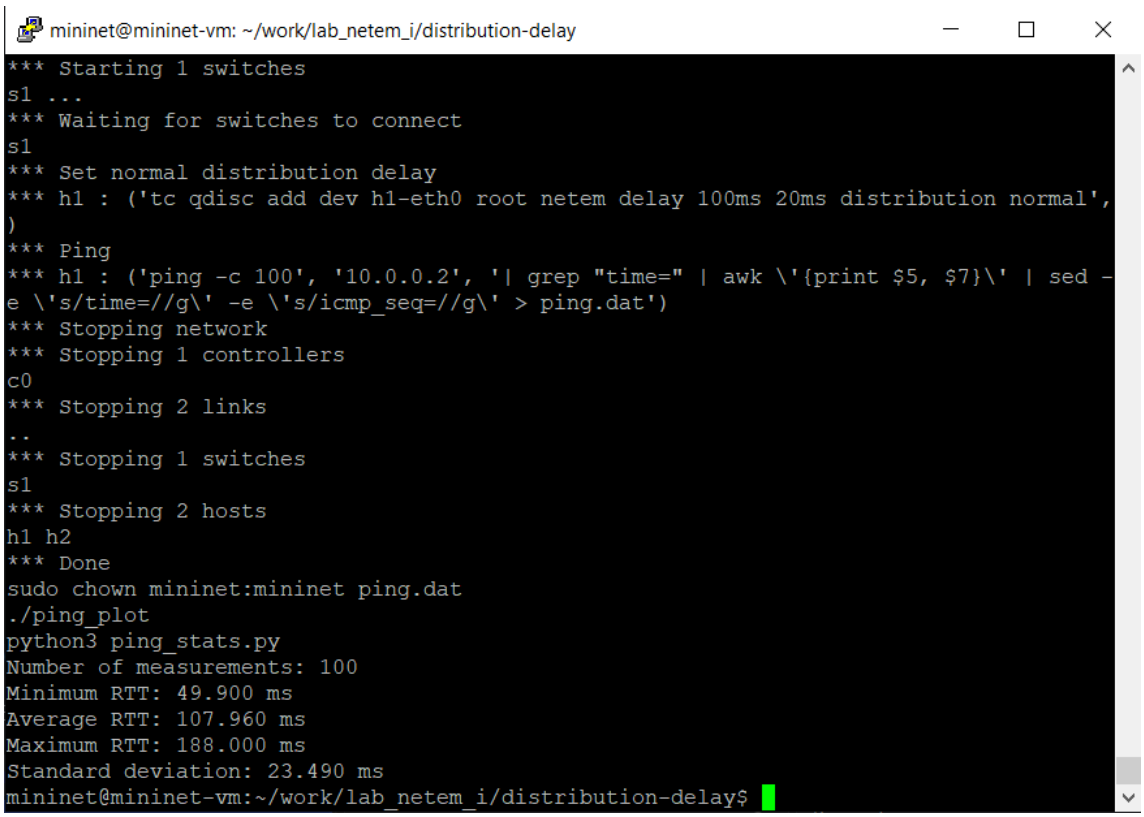


Рисунок 40: Результаты измерений с нормальным распределением

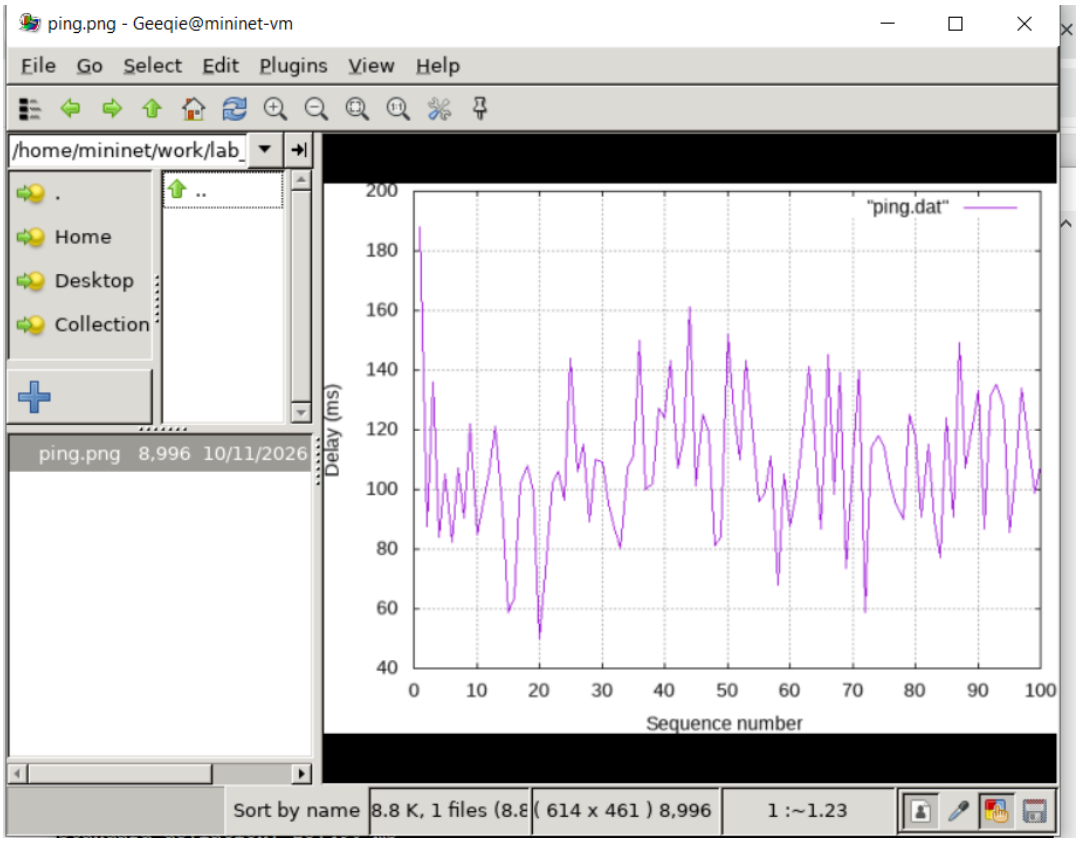


Рисунок 41: График RTT при нормальном распределении задержки

В опыте с корреляцией `avg=108,485` мс, σ=`19,297` мс. В опыте с нормальным распределением `avg=107,960` мс, σ=`23,490` мс. Здесь **25 % — параметр NETEM**, а не эмпирически вычисленный коэффициент корреляции; по одному графику также нельзя доказать соответствие наблюдений нормальному распределению.

7. Повторный запуск и итоговая статистика

После выполнения `make clean` в каталоге `simple-delay` был повторно выполнен `make`. Файлы `ping.dat` и `ping.png` успешно восстановлены; новая серия содержала 100 измерений.

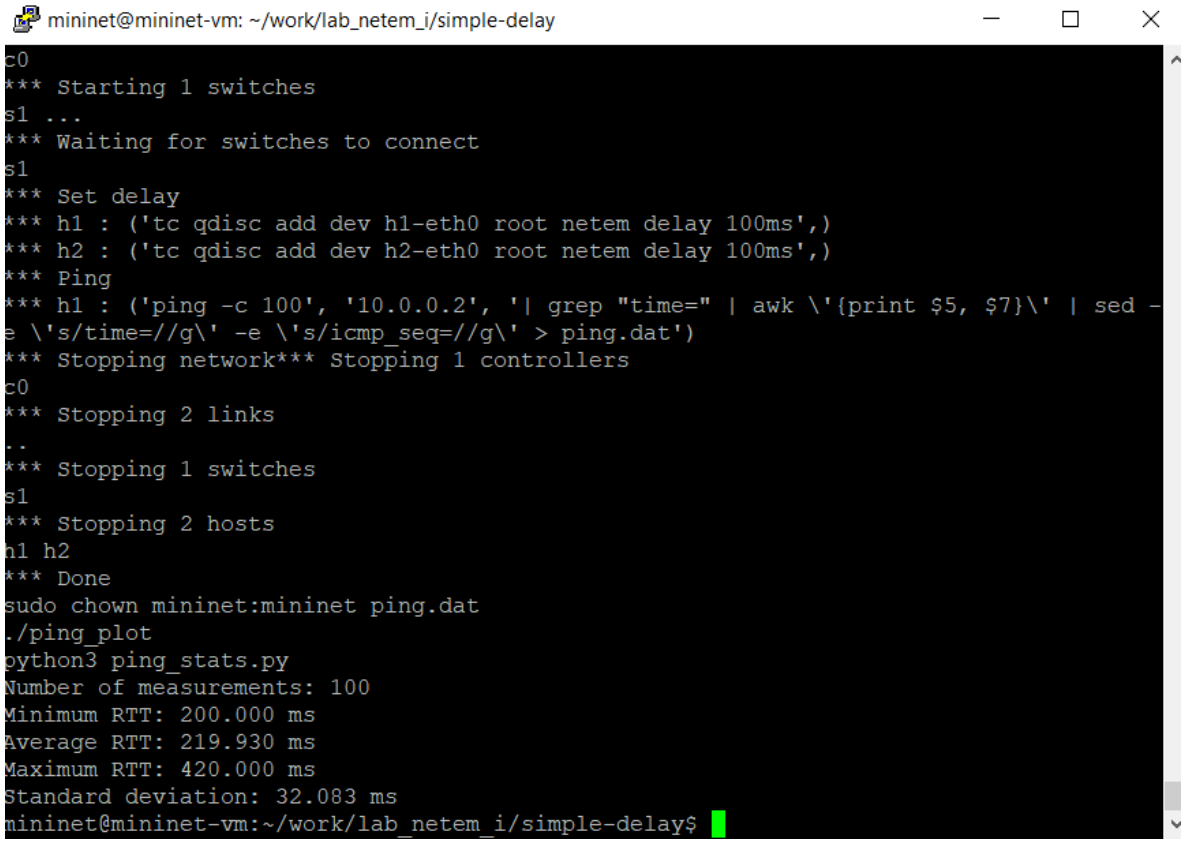


Рисунок 42: Повторный запуск simple-delay после очистки: 100 измерений RTT

Результаты воспроизводимых экспериментов

Эксперимент	N	Min, мс	Avg, мс	Max, мс	σ, мс
`simple-delay` (повторный запуск)	100	200,000	219,930	420,000	32,083
`change-delay`: 100+100 мс	100	200,000	215,960	644,000	48,885
`change-delay`: 50+50 мс	100	101,000	109,640	185,000	13,124
`jitter-delay`: 100 ± 10 мс	100	91,700	107,499	227,000	16,206
`correlation-delay`: 25 %	100	90,800	108,485	243,000	19,297
`distribution-delay`: normal	100	49,900	107,960	188,000	23,490

Организация файлов экспериментов

```
labs/lab04/project/
├── simple-delay/
├── change-delay/
├── jitter-delay/
```

```
├── correlation-delay/
└── distribution-delay/
```

Каждый каталог содержит `lab_netem_i.py`, `Makefile`, `ping_plot`, `ping_stats.py`, сохранённые `.dat`-файлы и `ping.png`. Для `change-delay` предусмотрены два файла данных. Полные исходные тексты программ и настройки доступны в каталоге проекта ЛР-4, включённом в материалы сдачи. Для повторного запуска внутри нужного каталога используется `make`; для удаления только результатов — `make clean`.

Выводы

В ходе работы создана виртуальная сеть из двух хостов и одного коммутатора, выполнены интерактивные и воспроизводимые эксперименты с NETEM. Измерения показали, что постоянная односторонняя и двусторонняя задержки отражаются в RTT: настройка по 100 мс на каждом хосте дала средний RTT около 205 мс в интерактивном опыте, а изменение до 50 мс — около 115 мс.

Воспроизводимый двухфазный эксперимент подтвердил снижение среднего RTT с **215,960** до **109,640 мс** при изменении задержки 100+100 до 50+50 мс. Введение джиттера, корреляции и нормального распределения привело к колебаниям RTT; дополнительную изменчивость могла вносить загрузка виртуальной машины. Реализованные Python-скрипты, Makefile и Gnuplot позволили повторно получить файлы измерений, графики и статистические показатели.

Список литературы

1. Королькова А. В., Кулябов Д. С. *Моделирование сетей передачи данных. Лабораторная работа № 4. Эмуляция и измерение задержек в глобальных сетях*. Методические указания, разделы 4.1–4.5.
2. Linux `tc-netem` — справочное руководство.
3. Mininet — документация и примеры.

Примечание о воспроизводимости иллюстраций. Все 41 изображение находятся в каталоге `image/` и включены в комплект отчёта. При переносе `.md/.qmd` вместе с этой папкой относительные пути к иллюстрациям сохраняются. Отдельный ZIP со скриншотами не нужен для конвертации.